Código explicado, con referencias al estado actual

Las referencias apuntan a riego_simple/main.py. Se revisaron líneas en el archivo actual; si se edita, hay que volver a consultar los números. El recorrido está deliberadamente en un solo archivo y sus encabezados numerados ayudan a leerlo de arriba hacia abajo.

Configuración — L16–39

Constantes: rutas (`CARPETA`, `ARCHIVO_DATOS`, `CARPETA_RESULTADOS`, `ORIGEN`), corte 28-06-2025 06:00, MADP `{1:40,2:40,3:60,4:80,5:70}`, caudal `{1:1.1,2:1.1,3:1.1,4:1.1,5:0.4}`, etiquetas, Base `[1,2,2,0,1,2,0,0,1]` y parámetros GA (población 30, generaciones 50, mutación .08, semilla 42).

Entrada: no aplica; son parámetros globales. Salida: configuración usada por preparación, división, motor y GA. Concepto: parametrización reproducible. Pregunta: “¿Qué representa un gen?” Respuesta: el índice del conjunto de salida asignado a una regla (0 Baja, 1 Media, 2 Alta), no una regla completa ni un peso.

Solapamiento — `resumir_riego_ventana`, L45–60

Recibe tabla de eventos, inicio de ventana y sector. Recorre eventos, conserva solo `standard_irrigation` del sector, calcula intersección entre evento y `[inicio,inicio+24h)`, cuenta los de duración solapada positiva y acumula minutos por caudal nominal. Devuelve cantidad de eventos y litros nominales. La llama `preparar_datos`. Concepto: construcción del objetivo mediante intervalo y caudal. Pregunta: “¿Y si atraviesa la frontera?” Respuesta: solo cuenta la parte solapada.

Preparación — `preparar_datos`, L62–123

Lee eventos y cada archivo `dataset_zone_{sector}.csv`; conserva humedad válida 0–100; desplaza `ts` diez minutos porque representa inicio de promedio; encuentra la observación disponible más reciente antes de las 06:00 (tolerancia 8 h); omite datos faltantes y decisiones con válvula abierta. Calcula último fin estándar, días transcurridos con máximo 30, humedad menos MADP, número de eventos y litros de ventana. Ordena sector-fecha y escribe `datos_riego.csv`.

Entrada: archivos procesados originales. Salida: una fila por sector-día disponible. En ejecución normal no se invoca salvo `--preparar`. Pregunta: “¿Incluye fertirrigación?” No, el resumen del target filtra eventos estándar.

División temporal — `leer_y_dividir_datos`, L128–157

Lee el CSV preparado; verifica columnas, faltantes, medición posterior a decisión, sectores, volúmenes negativos y coherencia entre eventos y litros. TRAIN usa fechas anteriores al 27 de junio; ese día se purga. TEST comienza 28 de junio. Devuelve dos tablas completas y cantidad purgada. La llama `ejecutar` antes del filtro de positivos. Concepto: holdout cronológico y prevención de fuga por etiqueta de 24 h. Pregunta: “¿Por qué no filtrar antes?” Porque primero se protege el corte usando toda la cronología, incluidos ceros.

Pertenencias — `tres_grados`, L162–169

Recibe arreglo numérico y puntos bajo/centro/alto. Calcula rampa Baja, rampa Alta y Media como residuo; devuelve una matriz cuyas columnas son grados Baja/Media/Alta. No son probabilidades. La usan `fuzzificar` y `preparar_salida`.

`fuzzificar`, L171–177: recibe filas con las dos entradas; calcula 3 grados de humedad con (-20,0,20), 3 de días con (0.5,1.5,4.5), combina todas las parejas usando mínimo y aplana a nueve activaciones. Devuelve grados de humedad, grados de días y nueve activaciones. Concepto: fuzzificación y AND de cada regla.

Conjuntos de salida — `preparar_salida`, L179–189

Recibe exclusivamente litros positivos de TRAIN. Calcula Q25/Q50/Q75, rechaza cuantiles no estrictos, construye universo lineal entre 0 y el máximo TRAIN con 501 puntos, y aplica la misma función de tres grados. Devuelve universo, tres conjuntos de salida y los tres cuantiles. La llama `ejecutar` solo después de separar cronológicamente. Pregunta: “¿Se usan litros TEST para fijar salida?” No.

Motor Mamdani — `inferir_mamdani`, L194–215

Entradas: activaciones `(casos,9)`, cromosoma de nueve valores ternarios, universo y conjuntos de salida. Verifica genes; para cada término Baja/Media/Alta toma el máximo de activación entre reglas que lo concluyen; trunca el conjunto con mínimo; agrega términos con máximo; calcula centroide discreto `Σ(μ*y)/Σμ`. Devuelve un litro estimado por caso. Lo llama el GA para evaluar TRAIN y `ejecutar` para Base/GA en TEST.

La orden de cada gen y la etiqueta humana están definidas por `REGLAS_BASE` y `ETIQUETAS_SALIDA` (L28–34). Implicación=min, agregación=max. Si el área es cero, `divide` entrega cero. Pregunta: “¿Qué produce antes del centroide?” Una función agregada sobre la malla de litros.

Fitness y mutación — L220–239

`calcular_mae` (L220–222) calcula media del valor absoluto del error. `calcular_fitness` (L224–231) evalúa el cromosoma por el motor y devuelve `1/(1+MAE)`; mayor fitness significa menor MAE, solo TRAIN. `mutar` (L233–239) copia el cromosoma, prueba cada gen contra 0.08 y a los mutados suma aleatoriamente 1 o 2 módulo 3: por tanto cambia a otro valor válido, nunca queda igual.

Bucle genético — `ejecutar_ga`, L241–279

Recibe activaciones y litros reales TRAIN, malla y conjuntos. Inicializa población 30×9; usa semilla 42; calcula fitness; conserva mejor histórico y anota historial; para cada transición copia el mejor (elitismo), elige padres por ruleta `fitness/fitness.sum()`, escoge corte 1–8, forma hijo concatenando prefijo/sufijo, muta; repite hasta completar población. Evalúa generación 0 y 50 (51 filas del historial). Devuelve mejor cromosoma y DataFrame histórico. Pregunta: “¿Qué cambia el GA?” Solo consecuentes de nueve reglas.

Evaluación y tabla de reglas — L284–307

`evaluar_modelo` calcula MAE, RMSE como raíz del promedio de error cuadrático y MedAE como mediana de error absoluto. `tabla_reglas` recorre combinaciones humedad×días, calcula número `3*h+d`, traduce etiquetas y forma CSV legible con Base y GA. Se llaman en `ejecutar`.

Consulta — `consultar_caso`, L309–338

Recibe sector, humedad observada, días, cromosoma y conjuntos de salida. Obtiene MADP, calcula humedad relativa, limita días a 30, reutiliza `fuzzificar`, lista reglas activadas y consecuentes Base/GA, y llama a Mamdani con ambas configuraciones. Devuelve grados, reglas y dos estimaciones condicionales. No genera una decisión binaria.

Programa principal — `ejecutar`, L343–451 y CLI L453–459

Primero, si se pidió, prepara datos. Luego divide cronología; después filtra litros positivos dentro de TRAIN y TEST. Prepara salida con TRAIN, ejecuta GA con TRAIN, y solo entonces calcula predicciones en TEST para Base, GA y baselines de media/mediana TRAIN. Genera predicciones globales, tabla por sector, imprime resumen y guarda cinco CSV. Opcionalmente valida argumentos y consulta un caso manual.

El bloque `if __name__ == "__main__"` crea opciones `--preparar`, `--sector`, `--humedad`, `--dias` y pasa argumentos a `ejecutar`. Para la consulta se requieren los tres argumentos de consulta juntos.

Pregunta frecuente: “¿Mamdani decide si regar?” No; tanto nombre de función como salida son estimaciones en litros condicionales. Los comentarios y docstrings importantes del archivo están en español. Los nombres de columnas y de eventos conservan vocabulario técnico/original para interoperabilidad.

Rastro de resultados

- `resultados/reglas.csv`: nueve consecuentes.
- `historial_ga.csv`: generaciones 0–50, mejor fitness/MAE TRAIN y promedio.
- `predicciones.csv`: target y cuatro predicciones por positivo TEST.
- `metricas.csv`: métricas globales.
- `metricas_por_sector.csv`: casos y errores por sector.

Preguntas de defensa rápidas

1. ¿Dónde está la función difusa? `main.py:L162–169`.
2. ¿Dónde se define el split? `L128–157`.
3. ¿Dónde se aplica el centroide? `L194–215`.
4. ¿Dónde se define mutación? `L233–239`.
5. ¿Dónde se ejecuta el flujo? `L343–459`.